

Лекция 8.

Централизованный мониторинг Linux-инфраструктуры: Zabbix

Цель лекции

Понять назначение мониторинга Linux-инфраструктуры

Разобрать базовую архитектуру Zabbix Server, Agent 2, Database и Frontend

Понять цепочку Host → Item → Trigger → Problem → Recovery

Подключить Linux-сервер web01 к Zabbix и получить метрики

Смоделировать отказ, увидеть Problem, выполнить Acknowledge, исправить и подтвердить Recovery

Учебные вопросы

1. Мониторинг, журналирование и диагностика
2. Zabbix Server, Agent, Database, Frontend и обзор Proxy
3. Host, Template, Item, Trigger, Problem и Severity
4. Graphs, Dashboard, Acknowledge и Maintenance
5. CPU, RAM, Disk, Network и Service Monitoring
6. Active и Passive Checks
7. Простой UserParameter, Notification и базовая безопасность

Главная модель Zabbix должна быть понятна первой

Host → Agent → Item → Value

Value → Trigger → Problem

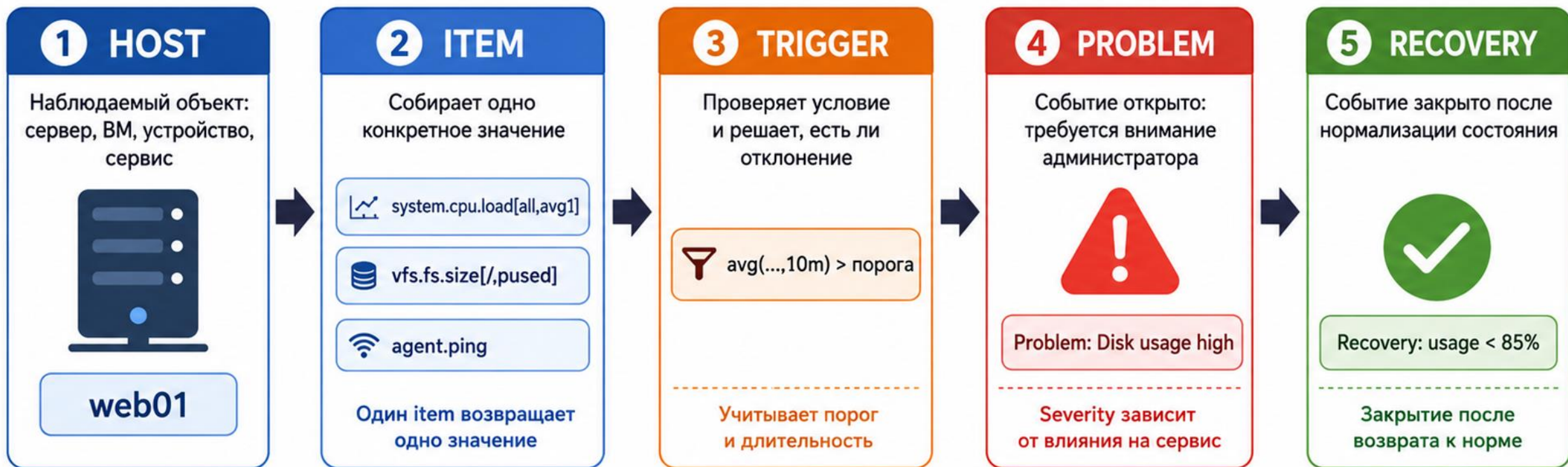
Problem → Administrator → Fix → Recovery

Host - наблюдаемый сервер или устройство

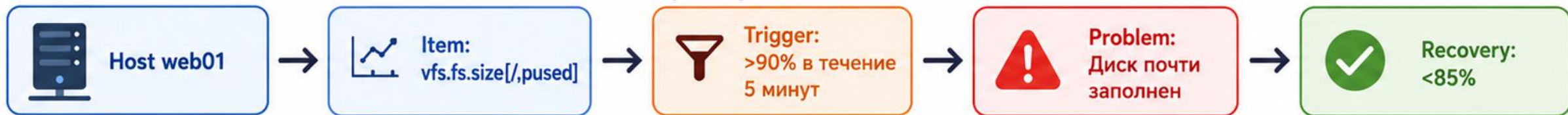
Item - конкретная метрика

Trigger - правило, которое превращает значение в Problem

Диаграмма: Host, Item, Trigger, Problem и Recovery



Пример логики



Ключевая идея



Host содержит items



Trigger превращает значение item в состояние Problem



Recovery закрывает проблему после нормализации

1. Мониторинг отвечает на вопрос: есть ли проблема сейчас

Monitoring - есть ли сейчас проблема?

Logging - что произошло?

Audit - кто выполнил действие?

Diagnostics - почему это произошло?

Zabbix обнаруживает симптом, но не заменяет системную диагностику

После Problem администратор все равно проверяет службу, журналы, сеть и приложение

Zabbix должен помогать реагировать, а не просто шуметь

Хороший мониторинг показывает важные отклонения

Плохой мониторинг создает десятки бесполезных тревог

Problem должен вести администратора к проверке причины

Recovery должен подтверждать, что условие нормализовалось

Цель лаборатории - увидеть полный цикл: Fault → Problem → Acknowledge → Fix → Recovery

2. Архитектура Zabbix состоит из нескольких ролей

Zabbix Agent 2 получает локальные метрики на Linux Host

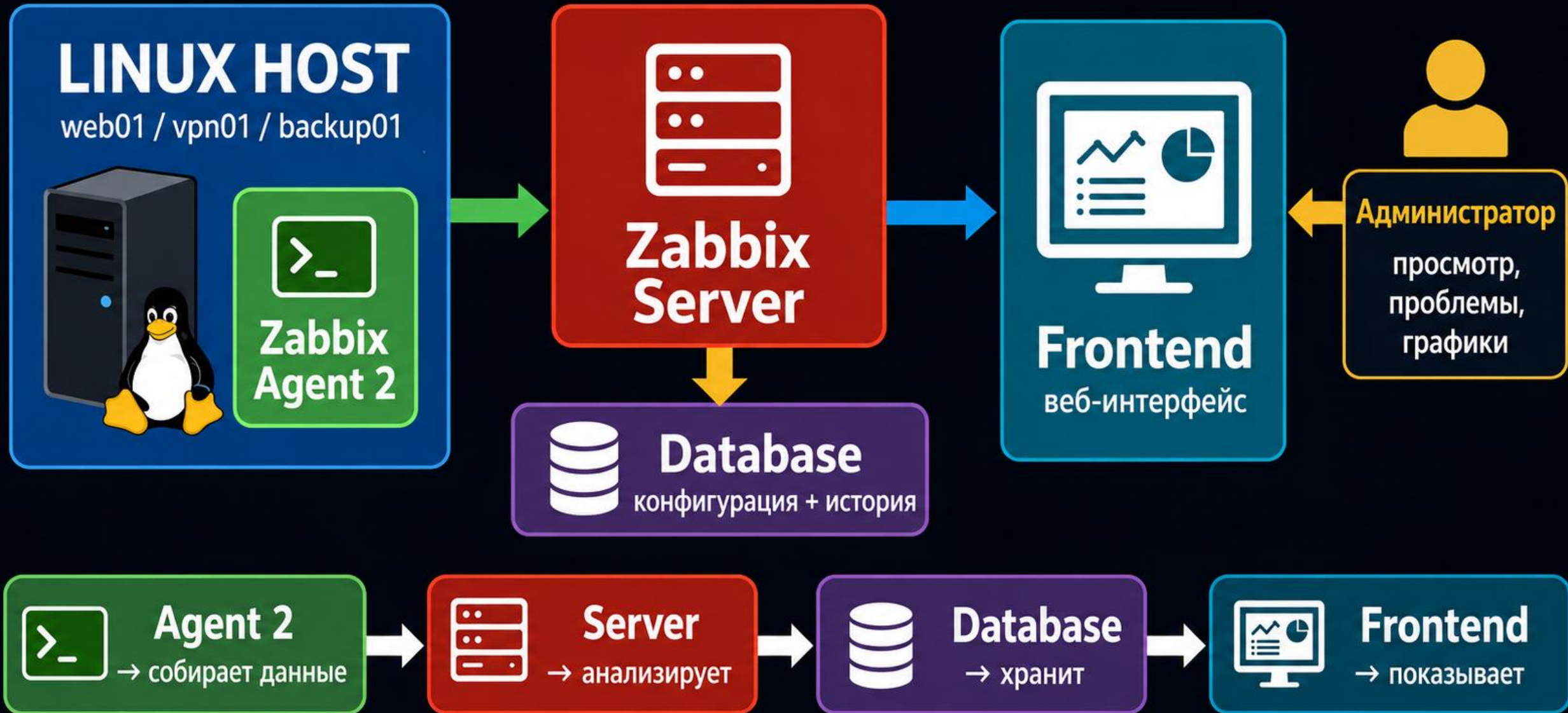
Zabbix Server собирает, анализирует значения и создает Problems

Database хранит конфигурацию, историю и события

Frontend показывает состояние через web-интерфейс

Administrator видит Problems, графики, dashboards и историю восстановления

Архитектура Zabbix: Host, Agent 2, Server, Database и Frontend



Zabbix Proxy нужен не в первой лаборатории

Proxy собирает данные от узлов филиала или отдельной зоны

Proxy применяют на филиалах, в DMZ и крупных инфраструктурах

Branch Hosts → Zabbix Proxy → Zabbix Server

Proxy помогает при слабой связи и распределенной инфраструктуре

В основной лаборатории Proxy не устанавливается

В лаборатории используем Zabbix Agent 2

Agent 2 - основной агент лаборатории

Файл конфигурации: `/etc/zabbix/zabbix_agent2.conf`

Service: `systemctl status zabbix-agent2`

Classic Agent можно упомянуть обзорно

Важно заранее выбрать один агент, чтобы не смешивать конфигурации

Базовая конфигурация Agent 2

Server=192.168.30.40

ServerActive=192.168.30.40

Hostname=web01

sudo systemctl restart zabbix-agent2

sudo systemctl status zabbix-agent2

После изменения конфигурации всегда проверяют
состояние службы Agent 2

Passive Check инициирует Zabbix Server

Passive → Server инициирует запрос

Zabbix Server обращается к Agent TCP 10050

Agent возвращает значение item

Для проверки можно использовать `zabbix_get`

Пример: `zabbix_get -s 192.168.30.10 -k system.hostname`

Active Check инициирует Agent

Active → Agent инициирует подключение

Agent подключается к Zabbix Server TCP 10051

Agent получает список active checks и отправляет значения

Active checks полезны, когда Server не может напрямую опросить Agent

Не нужно перегружать первую лабораторию сложными сетевыми исключениями

Hostname особенно важен для Active Checks

Agent: Hostname=web01

Frontend: Host name=web01

Имена должны совпадать

Visible name нужен только для удобного отображения

Если Hostname не совпал, active checks могут не привязаться к нужному host

3. Host, Template, Item и Trigger связываются в цепочку

Host → web01

Template → Linux checks

Item → disk usage

Trigger → disk > 90%

Problem → Disk usage high

Эта цепочка важнее запоминания всех возможностей
Zabbix

Item должен возвращать одно значение

Item → одна метрика → одно значение

Примеры: CPU load, RAM available, Disk usage

Примеры: Network traffic, Service status, Backup age

Item не должен возвращать многострочный отчет

Если нужен отчет, его делают отдельно, а в мониторинг отдают конкретную метрику

Supported, Unsupported и No Data показывают состояние проверки

Supported → значение приходит

Unsupported → проверка не может получить корректное значение

No Data → новые значения перестали приходить

Unsupported обычно проверяют по ошибке item и правам агента

nodata() как сложную trigger-функцию оставляем обзорно

Дополнительно: Low-Level Discovery автоматизирует массовые проверки

Zabbix умеет автоматически находить файловые системы и интерфейсы

Discovery Rule находит объекты

Item Prototype создает items по найденным объектам

Trigger Prototype создает triggers по найденным объектам

В основной лаборатории LLD не настраивается вручную

Trigger превращает значение Item в Problem

Item → число или статус

Trigger → правило

Problem → правило выполнилось

Пример: Disk Usage > 90%

Trigger должен ответить на вопрос: когда администратору нужно реагировать?

Trigger должен учитывать длительность проблемы

CPU 95% на 3 секунды обычно не авария

CPU 95% 10 минут может быть Problem

last() смотрит последнее значение

avg(), min(), max() помогают оценивать период

nodata() помогает обнаружить отсутствие новых значений

timeleft() переносим в дополнительный материал

Recovery threshold уменьшает постоянное мигание Problem

Problem $\rightarrow$ Disk $> 90\%$

Recovery $\rightarrow$ Disk $< 85\%$

Если порог открытия и закрытия одинаковый, Problem может часто открываться и закрываться

Раздельные пороги помогают стабилизировать уведомления

Термин hysteresis можно упомянуть обзорно, без глубокого разбора

Severity показывает важность, а не тип метрики

Severity - насколько проблема важна для эксплуатации

Nginx Down на test → Warning

Nginx Down на production → High

Одна и та же метрика может иметь разную важность в разных средах

Severity помогает расставлять приоритет реакции

4. Problem, Acknowledge, Recovery и Maintenance различаются

Problem → обнаружена проблема

Acknowledge → администратор взял ее в работу

Recovery → условие нормализовалось

Maintenance → запланированные работы

Manual Close не делаем обязательной темой этой лекции

Graphs и Dashboard помогают увидеть состояние, а не только тревоги

Graphs показывают изменение метрики во времени

Dashboard собирает важные виджеты на один экран

Для junior dashboard должен отвечать на простые вопросы

Работают ли основные hosts?

Есть ли Problems?

Какие ресурсы близки к порогам?

Tags используем только обзорно

service=nginx

env=lab

Tags помогают фильтровать Problems

Tags могут использоваться для маршрутизации Actions

Подробную маршрутизацию Actions по tags переносим
в дополнительный материал

5. Linux metrics не нужно объяснять заново глубоко

CPU → длительная высокая нагрузка

RAM → мало available

Disk → usage

Inode → usage

Network → traffic/errors

Service → active/inactive

Service Monitoring проверяет несколько уровней

Process → systemd Unit → Local Port

Local Port → Remote Port → Application Response

Unit Active ≠ Application работает

Служба может быть active, но не слушать нужный порт

Порт может отвечать, но приложение может
возвращать HTTP 500

Nginx на web01 проверяем как основной сервисный пример

1. nginx unit active
2. TCP 80 listening
3. TCP 80 reachable
4. HTTP response

Остановка Nginx должна вызвать Problem

Запуск Nginx должен привести к Recovery

OpenVPN monitoring оставляем обзорным примером

Проверять можно process, UDP 1194, logs и real VPN connection

UDP нельзя проверять как обычный TCP connect

OpenVPN connected не всегда доказывает доступ к внутреннему сервису

В этой лекции OpenVPN monitoring не становится отдельной лабораторией

Главная сервисная практика - Nginx на web01

Дополнительно: Dependencies уменьшают alarm storm

Router Down может вызвать десятки вторичных Problems

Dependencies помогают показать первопричину выше вторичных отказов

Это важно для крупных инфраструктур

В основной лаборатории Dependencies не настраиваются

Сначала студент должен понять один Host, Item, Trigger и Problem

Backup monitoring связываем с предыдущей лекцией

Главная метрика: `backup.age`

`backup.age` → возраст последней успешной копии

RPO = 24 часа

`backup.age` > 86400 → Problem

Дополнительно можно контролировать `duration` и `size`
`files_count`, `restore_test_age` и `anomaly detection` не
нужны в основной части

7. UserParameter добавляет простую пользовательскую метрику

UserParameter=custom.backup.age,/usr/local/sbin/backup-age-zabbix.sh

Скрипт должен вывести одно машинно читаемое значение

Для Numeric Item корректный пример вывода: 43200

Неправильно для Numeric Item: OK: backup age is 12 hours

Numeric Item должен получать число без лишнего диагностического текста

UserParameter возвращает значение через вывод команды

Значение Item формируется из вывода команды

UserParameter

Exit code не становится автоматически значением Item

Для Numeric Item команда должна вернуть корректное числовое значение

Лишний текст может сделать item Unsupported

Диагностические сообщения лучше писать в log, а не в значение item

Права Agent нужно проверять от имени пользователя zabbix

`sudo -u zabbix /usr/local/sbin/backup-age-zabbix.sh`

Работает от root ≠ работает из Zabbix

Пользователь zabbix должен иметь только
минимально нужные права

Нельзя выдавать zabbix правило NOPASSWD: ALL

Если item Unsupported, сначала проверяют права, путь
к скрипту и вывод команды

Тяжелые проверки нельзя выполнять синхронно из Agent

UserParameter должен выполняться быстро

Не запускать внутри polling: du /

Не запускать внутри polling: find /

Не запускать внутри polling: backup или restore

Для тяжелых задач: background job → state file →

Zabbix читает готовое значение

Notification связывает Problem с реакцией администратора

Problem → Action → Notification

Примеры каналов: Email или Webhook

Recovery notification сообщает, что условие нормализовалось

Escalation переносим в дополнительный материал

Для junior важно понять, какое событие должно прийти администратору

Базовая безопасность Zabbix без перегруза

Agent должен доверять только нужным Server или Proxy

TCP 10050 не открывать всему Интернету

Frontend должен работать через HTTPS

Credentials должны иметь минимальные права

Secrets не выводить в Items, Logs, trigger names и notifications

TLS PSK и certificates для Agent рассматриваются обзорно

Secret Macros не являются полноценным vault сами по себе

Secret text скрывает значение в интерфейсе

Vault secret получает значение из внешнего хранилища

Secret Macro сама по себе не делает секрет

безопасным во всех местах

Нельзя выводить секреты в item values

Нельзя выводить секреты в trigger names, logs и notifications

Сам Zabbix тоже нужно мониторить обзорно

Zabbix Server process

Database

Disk Space

Queue

poller busy, preprocessing busy, cache и housekeeper

переносим в дополнительный материал

Backup самого Zabbix также относится к отдельной эксплуатационной теме

Центральная лабораторная топология

Zabbix Server: 192.168.30.40

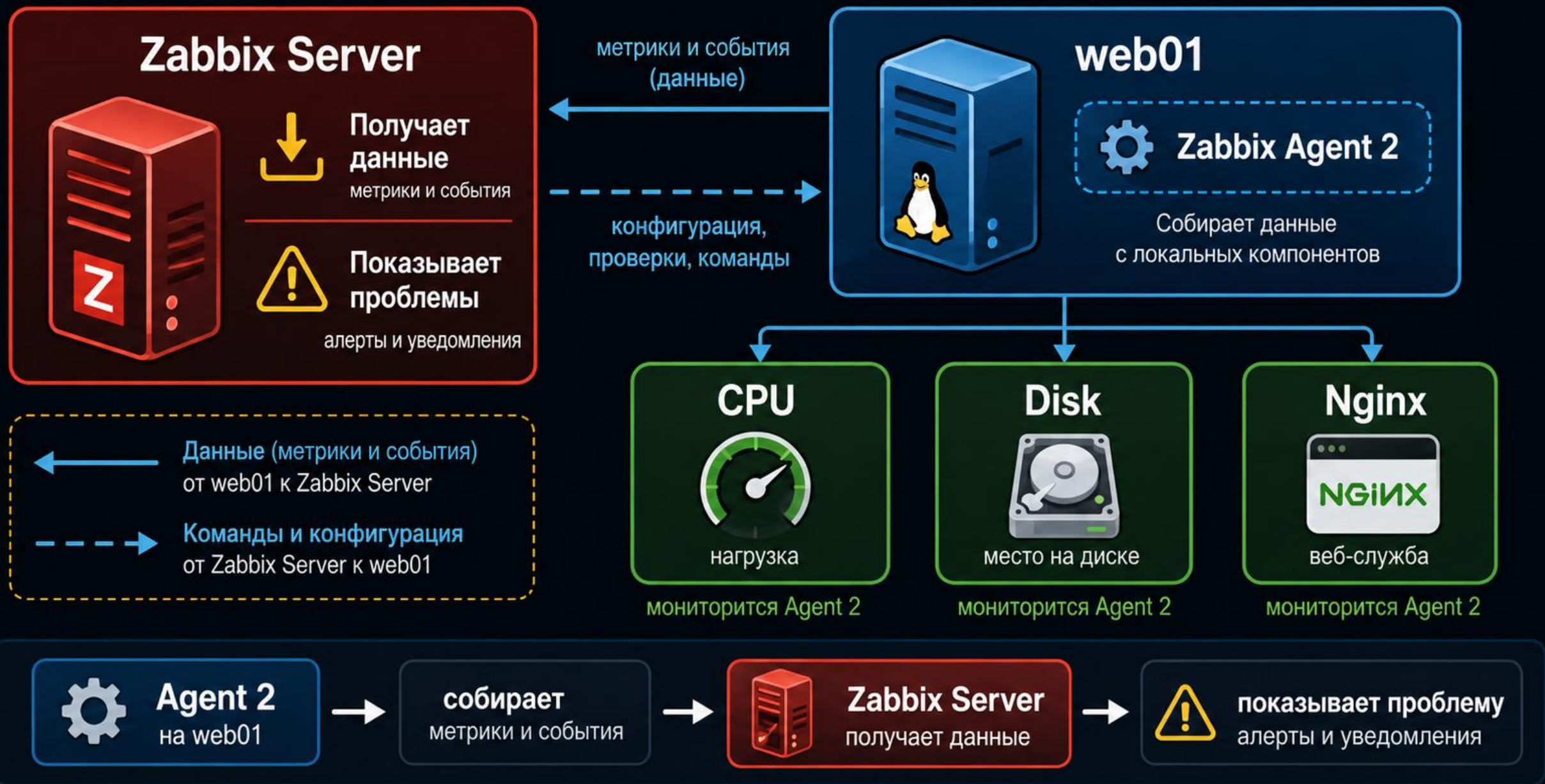
web01: Zabbix Agent 2

Метрики web01: CPU, Disk и Nginx

Nginx проверяется как service и HTTP

vpn01 и backup01 используются как дополнительные
сценарии после основной лаборатории

Топология Zabbix: Server, web01 Agent 2, CPU, Disk и Nginx



Основной лабораторный цикл: подключение web01

1. Установить Agent 2
2. Настроить Server, ServerActive и Hostname
3. Добавить web01 во Frontend
4. Привязать Linux Template
5. Убедиться в поступлении данных
6. Создать или проверить Nginx monitoring

Основной лабораторный цикл: Problem и Recovery

7. Остановить Nginx
8. Получить Problem
9. Выполнить Acknowledge
10. Запустить Nginx
11. Получить Recovery
12. Заполнить test filesystem выше порога и получить Disk Problem
13. Очистить тестовый файл и получить Recovery
14. Добавить простой backup.age UserParameter

Матрица отказов делает лабораторию проверяемой

Agent остановлен → Agent unavailable

Nginx остановлен → Service down

HTTP 500 → Application problem

Test FS > 90% → Disk high usage

Backup старше RPO → Backup stale

Цикл для каждого отказа: Fault → Problem →

Acknowledge → Fix → Recovery

Цикл: monitoring → problem → acknowledge → recovery



Главный вывод

Мониторинг обнаруживает симптом, администратор подтверждает проблему, устраняет причину и проверяет Recovery.

Дополнительный материал

Proxy, Low-Level Discovery, timeleft() и подробный hysteresis

Tags routing, Dependencies и flexible UserParameters
sudo wrappers и heavy-check state architecture

Escalation и подробная TLS-конфигурация Agent

Vault integration, внутренние метрики

производительности Zabbix и backup самого Zabbix

Итоги лекции

Zabbix нужен, чтобы вовремя обнаружить симптом и показать его администратору

Главная цепочка: Host → Item → Trigger → Problem → Recovery

Agent 2 на web01 передает метрики в Zabbix Server и Frontend

Nginx нужно проверять не только как systemd unit, но и как порт и HTTP-ответ

UserParameter должен возвращать одно машинно читаемое значение

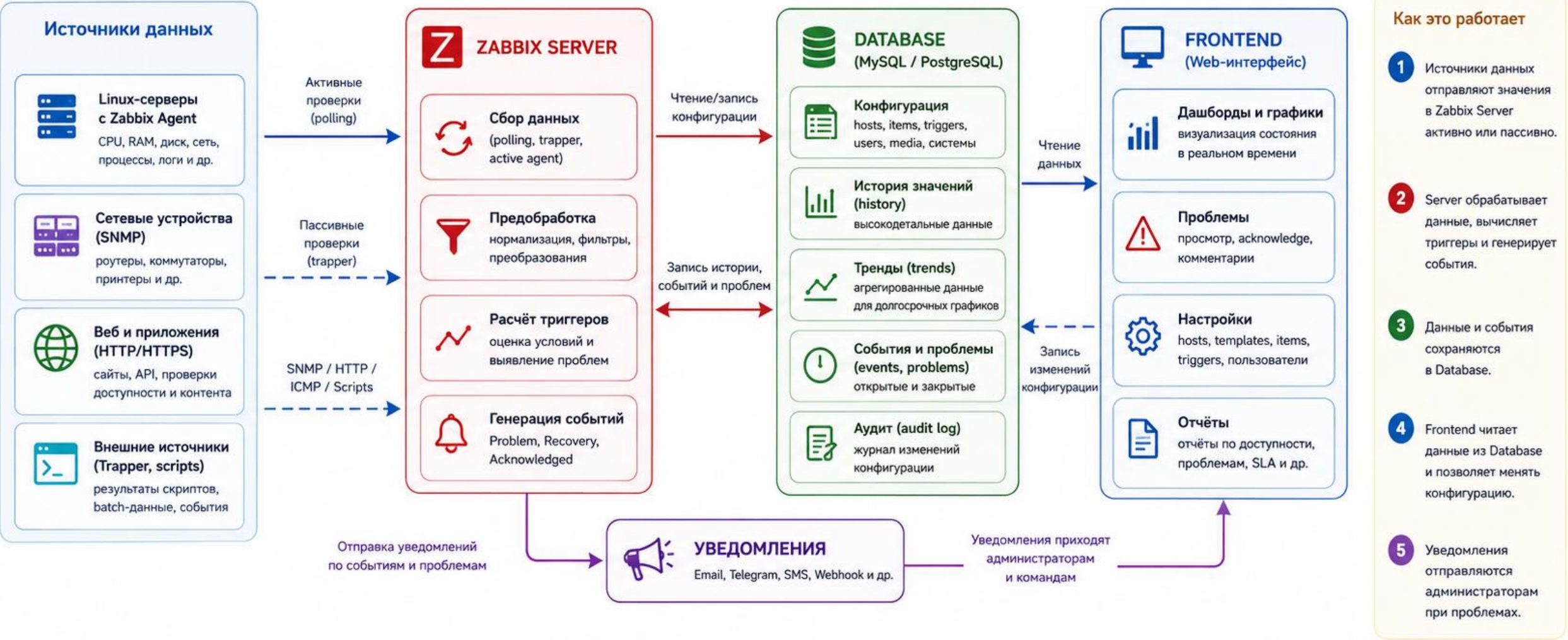
Результат лаборатории - реальный Problem, Acknowledge, Fix и подтвержденный Recovery

Контрольные вопросы

1. Для чего нужен мониторинг?
2. Чем мониторинг отличается от журналирования?
3. Для чего нужны Zabbix Server и Agent?
4. Чем Active Check отличается от Passive Check?
5. Что такое Host?
6. Что такое Item?
7. Что такое Trigger?
8. Что такое Problem и Recovery?
9. Почему systemd active не доказывает работу Web-приложения?
10. Как проверить, что исправленная проблема действительно закрылась?
11. Для чего используется UserParameter?

Схема: поток данных Zabbix Agent, Server, Database и Frontend

Как метрики попадают от устройств в интерфейс и превращаются в проблемы и уведомления



Ключевые принципы



Единая точка обработки
Вычисление триггеров происходит только на Zabbix Server.



Надёжное хранение
Database хранит историю, тренды, события и конфигурацию.



Frontend не собирает данные
Он только отображает и изменяет конфигурацию через Database.

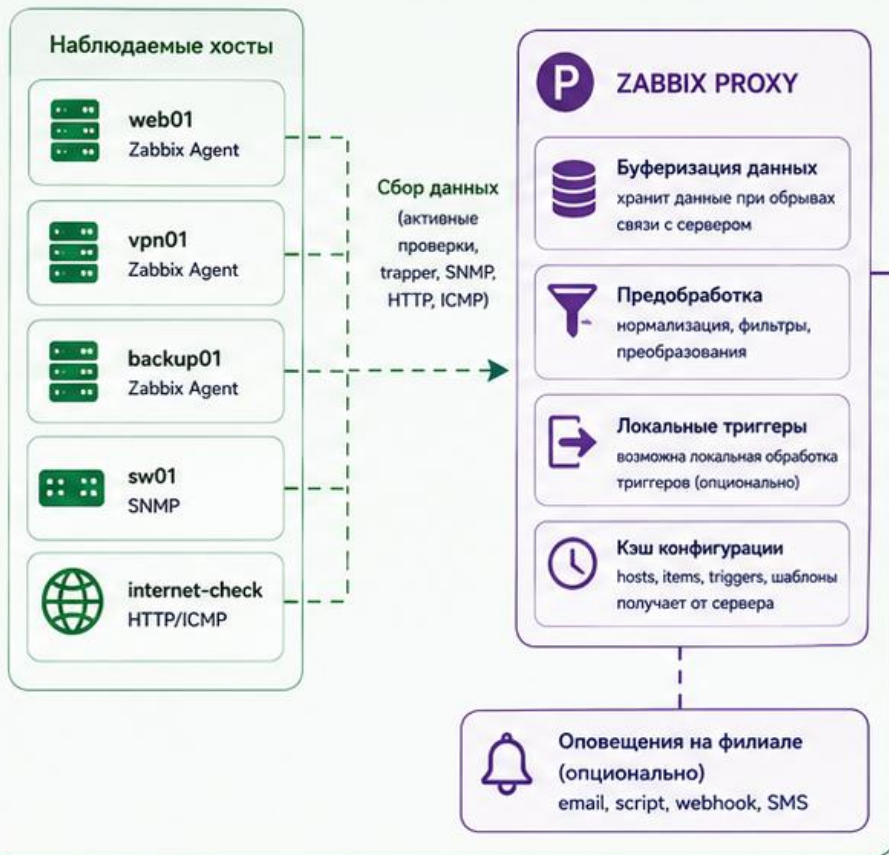


Безопасность и доступ
Доступ к Frontend защищён, а агенты используют ключи.

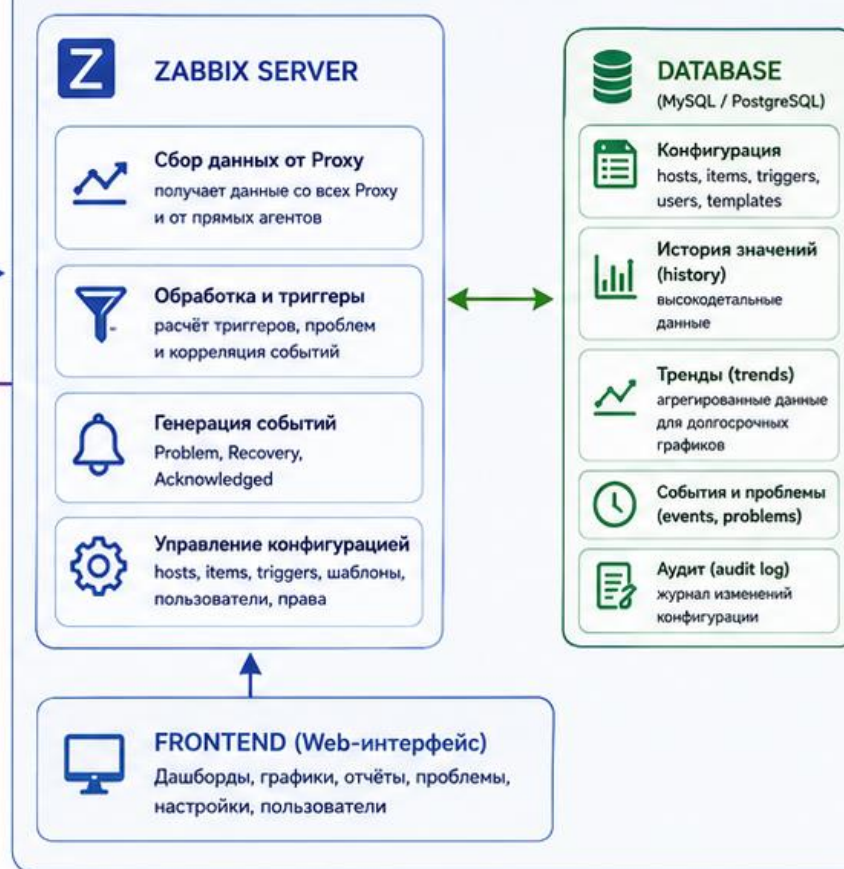
Схема: Zabbix Proxy между филиалом и центральным сервером

Proxy собирает данные на стороне филиала и передаёт их на Zabbix Server. Server управляет и хранит всё в базе данных.

ФИЛИАЛ / УДАЛЁННАЯ ПЛОЩАДКА



ЦЕНТРАЛЬНЫЙ СЕРВЕР / ДАТА-ЦЕНТР



Как это работает

- 1 Агенты в филиале отправляют данные на Zabbix Proxy (локальный сбор).
- 2 Proxy буферизирует и преобразовывает данные, хранит кэш конфигурации.
- 3 Proxy передаёт данные на Zabbix Server через исходящее соединение (TCP 10051).
- 4 Server обрабатывает данные, вычисляет триггеры и сохраняет в Database.
- 5 Frontend читает данные из Database и предоставляет их администраторам.

Важно!
Proxy не требует входящих соединений из дата-центра. Нужен только исходящий канал к Zabbix Server.

Зачем нужен Proxy



Снижение нагрузки на канал
Передаёт агрегированные данные, а не каждый запрос.



Работа при обрывах связи
Данные сохраняются в буфере и отправляются позже.



Безопасность
Server не обращается напрямую к устройствам в филиале.



Масштабирование
Поддерживает тысячи хостов на удалённых площадках.



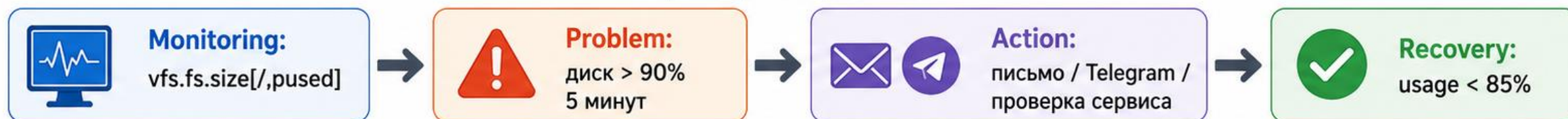
Изоляция сети
Подходит для DMZ и сложных сетевых схем.

Диаграмма: цикл monitoring, problem, action и recovery

Как мониторинг превращает отклонение в действие администратора и возвращается к нормальному состоянию



Пример на практике



Ключевая идея

Monitoring выявляет отклонение, Problem фиксирует его, Action запускает реакцию, Recovery закрывает событие и цикл продолжается.